

Tổng quan tài liệu

MOVIE CAFÉ BOOKING SYSTEM

Website đặt phòng café phim, chọn phim trước, gọi món và quản lý phục vụ tại quầy.

Thuộc tính	Nội dung
Phiên bản	1.7 – Bản chốt: nội dung 1.2 sau hai vòng phản biện; phân công, lịch 1 tuần; giao diện nhiều trang; Phụ lục C; kho phim thật
Ngày lập	07/09/2026 (cập nhật 14/09/2026)
Đối tượng đọc	Nhóm 6 sinh viên năm cuối ngành Công nghệ thông tin; giảng viên chấm đồ án
Thời gian thực hiện	1 tuần (lịch 7 ngày ở trang 16), có AI hỗ trợ; đồ án cuối kỳ môn Phát triển web; nhắm mức Advanced
Công nghệ	HTML5, CSS3, TypeScript + Vite (nhiều trang) – Node.js 24 LTS/Express – PostgreSQL 16 + Prisma; Locust (Python) đo tải
Mô hình triển khai	Một website, một cơ sở dữ liệu; phân quyền khách hàng, nhân viên và quản lý
Phạm vi trọng tâm	Một cơ sở; đặt phòng theo gói 2 hoặc 3 giờ; chọn phim tùy chọn; gọi món; thanh toán tại quầy

Mục đích, căn cứ và giả định

Tài liệu là căn cứ chia việc, thiết kế giao diện, xây dựng cơ sở dữ liệu, kiểm thử và nghiệm thu. Mục ghi “bắt buộc” là phạm vi final; mục “mở rộng” chỉ làm khi luồng chính đã ổn định và cả nhóm đồng ý trước mốc đóng băng tính năng. Lấy cảm hứng từ mô hình café phim phòng riêng của T-Box; đây là thiết kế đồ án độc lập, không phải đặc tả nội bộ của T-Box. Giá, giờ hoạt động và chính sách là giả định của nhóm cho bản demo. Quy tắc đã chốt: không đặt cọc, không thanh toán online, không giữ chỗ tạm khi khách đang chọn, chưa gia hạn hoặc đổi lịch trực tiếp; chọn phim không phát sinh phí riêng.

Thay đổi so với bản 1.0 và các hiệu chỉnh 1.2–1.7

Hạng mục	Nội dung thay đổi
Cơ sở dữ liệu	Chốt PostgreSQL và Prisma. Chống trùng phòng bằng hai lớp: giao dịch có khóa bản ghi phòng và ràng buộc loại trừ (EXCLUDE) ở tầng cơ sở dữ liệu. Có mục riêng (trang 12).
Trạng thái booking	Tách trạng thái sử dụng phòng khỏi trạng thái thanh toán; kết thúc sớm có lý do để booking không kẹt ở IN_USE.
Xử lý ngoại lệ	Mục riêng (trang 8): giảm/miễn tiền, khách bỏ về chưa trả, booking quá hạn.
Đăng nhập	Chốt phiên đăng nhập phía máy chủ qua cookie; giới hạn số lần thử đăng nhập; khóa tài khoản thu hồi phiên ngay.
Kiểm thử nâng cao	Thêm mục riêng (trang 15): môi trường đo, công cụ, ba kịch bản, chỉ số, bảng trước/sau và tiêu chí kết luận. Bổ sung kịch bản nghiệm thu T15–T18.
Phân công, tiến độ	Bổ sung người phụ trách tích hợp và người phụ trách đo hiệu năng; đo từ tuần 3; đóng băng tính năng cuối tuần 3.
Hiệu chỉnh 1.2	Giảm/miễn tiền phải được quản lý duyệt trước khi áp dụng; quy tắc món khi kết thúc sớm; chống gửi lặp xử lý ngoại giao dịch chính; sửa kịch bản Locust; dữ liệu mẫu 12 tháng; schema bảng session; Node.js 24; check-in muộn và phim tại quán.
Hiệu chỉnh 1.3–1.7	Phân công theo module có tên (trang 16); hợp đồng ranh giới 3–5–6; lịch 7 ngày, giữ nguyên phạm vi; giao diện HTML5/CSS3/TypeScript nhiều trang với Vite; 1.6 Phụ lục C (giao diện, bộ nền); 1.7 phim thật (The Movies Dataset), danh mục phân trang.

Phạm vi và cách đọc

Trang	Nội dung
3	Vai trò, phân quyền, đăng nhập và kiến trúc
4	Phòng và quy tắc thời gian
5	Đặt phòng, trạng thái sử dụng và trạng thái thanh toán
6	Chọn phim trước và chuẩn bị phim
7	Đồ ăn, hóa đơn và thanh toán
8	Xử lý ngoại lệ tại quầy
9	Các luồng sử dụng chi tiết
10	Màn hình và yêu cầu giao diện
11	Mô hình dữ liệu PostgreSQL
12	Chống đặt trùng và giao dịch
13	API và nguyên tắc triển khai
14	Yêu cầu chất lượng và kịch bản nghiệm thu
15	Kiểm thử hiệu năng và tải
16	Phân công, tiến độ và bàn giao
17	Đánh giá đề tài, kế hoạch trình bày và Q&A
18–21	Phụ lục A: SQL ràng buộc và giao dịch; Phụ lục B: kịch bản Locust; Phụ lục C: cấu trúc giao diện nhiều trang

Năm nhóm chức năng chính

F1. Tra cứu phòng và chọn phim. **F2.** Đặt và quản lý booking. **F3.** Đặt món và hóa đơn. **F4.** Vận hành tại quầy, gồm xử lý ngoại lệ. **F5.** Quản trị và báo cáo. Đăng nhập và phân quyền là nền tảng dùng chung. Kiểm thử hiệu năng và tải là phần nâng cao có bằng chứng, không phải nhóm chức năng riêng.

Không thuộc phạm vi final

Nhiều chi nhánh, phát phim trực tuyến, điều khiển máy chiếu, tích hợp tài khoản nền tảng xem phim, ví điện tử hoặc ngân hàng thật, đặt cọc và hoàn tiền online, voucher, tích điểm, kho nguyên liệu, quản lý công nợ đầy đủ, ứng dụng di động, AI gợi ý phim, giữ chỗ tạm có hết hạn, giá theo khung giờ, gia hạn tại chỗ.

Thứ tự ưu tiên

P0: Đặt phòng đúng, chống trùng, phân quyền, tính tiền, hoàn thành booking và xử lý ngoại lệ tại quầy. **P1:** Chọn phim, món, giao diện nội bộ, báo cáo cơ bản, kiểm thử hiệu năng có số liệu; vẫn cần có trong bản final. **P2:** Email, trailer; không cam kết trong tháng đầu. Nếu chậm tiến độ, cắt P2 trước, không bỏ P0.

Nguyên tắc chung từ bản 1.1: sửa những gì giúp hệ thống đúng và kiểm chứng được; giữ phạm vi nhỏ để nhóm hoàn thành trong một tháng và demo ổn định.

Vai trò và kiến trúc

Một website có khu vực khách và khu vực nội bộ; cùng một booking đi xuyên suốt từ khách đặt đến nhân viên thanh toán.

Thao tác	Khách	Nhân viên	Quản lý
Xem phòng, phim, menu	Có	Có	Có
Đặt và xem booking cá nhân	Có	Có	Có
Xem toàn bộ lịch phòng, bộ lọc quá hạn	Không	Có	Có
Tạo booking tại quầy	Không	Có	Có
Check-in, món, thu tiền, kết thúc sớm	Không	Có	Có
Ghi nhận điều chỉnh tiền (chờ duyệt)	Không	Có	Có
Duyệt điều chỉnh, miễn tiền	Không	Không	Có
Sửa phòng, phim, menu, giá	Không	Không	Có
Quản lý nhân viên, báo cáo	Không	Không	Có

Đăng nhập, phiên và tài khoản

- Khách tự đăng ký bằng họ tên, email, số điện thoại, mật khẩu (băm bcrypt); hệ thống luôn gán vai trò CUSTOMER, không nhận vai trò từ trình duyệt. Quản lý tạo/khóa tài khoản nhân viên; tài khoản quản lý đầu tiên đến từ dữ liệu khởi tạo, không có đăng ký quản lý công khai.
- Phiên lưu phía máy chủ trong bảng session (schema connect-pg-simple sid, sess, expire + cột sinh user_id để tìm phiên theo người dùng); trình duyệt chỉ giữ cookie định danh phiên (HttpOnly, SameSite=Lax, Secure khi HTTPS). Không lưu phiên trong bộ nhớ tiến trình.
- Mọi yêu cầu được bảo vệ kiểm tra phiên còn hiệu lực và tài khoản còn hoạt động. Khóa tài khoản xóa toàn bộ phiên của tài khoản đó; yêu cầu kế tiếp bị từ chối ngay, không chờ hết hạn.
- Giới hạn thử đăng nhập: tối đa 5 lần sai/10 phút theo email hoặc IP; vượt trả 429 và ghi nhật ký.
- Một màn hình đăng nhập chung; khách vào trang đặt phòng, nhân viên và quản lý vào khu nội bộ, quản lý có thêm menu theo quyền.

Cấu trúc và công nghệ

/: website khách; /staff: lịch, booking, món, hóa đơn, ngoại lệ; /admin: danh mục, nhân viên, duyệt điều chỉnh, báo cáo. Giao diện theo **kiến trúc nhiều trang** (mỗi màn hình một HTML + một module TypeScript) để giảm phụ thuộc; thư mục dùng chung shared/ (gọi API, kiểm tra phiên, định dạng, bố cục, CSS) do người 1 giữ. Một backend thống nhất.

Tầng	Lựa chọn	Lý do
Giao diện	HTML5, CSS3, TypeScript; Vite (nhiều điểm vào HTML); không dùng framework SPA	Mỗi người sở hữu trọn trang của mình; ít phụ thuộc
Máy chủ	Node.js 24 LTS (Node 20 hết hỗ trợ 04/2026), Express, zod, express-session + connect-pg-simple, express-rate-limit, helmet	Một backend thống nhất; chốt phụ thuộc trong lockfile
Cơ sở dữ liệu	PostgreSQL 16 qua Prisma; Prisma Migrate, ràng buộc đặc thù viết tay trong migration	Giao dịch, khóa dòng và ràng buộc loại trừ khớp thiết kế chống trùng
Kiểm thử	Vitest + Supertest (API), Playwright (luồng đặt phòng), Locust – Python (tải)	Nhóm quen Python; một công cụ tải là đủ
Triển khai	Docker Compose để phát triển và đo; bản demo có HTTPS	Số liệu đo cần môi trường cố định

Kiểm soát truy cập: mọi API kiểm tra phiên, vai trò và quyền trên dữ liệu; khách chỉ xem/sửa booking của mình, đổi ID trên URL hay gọi thẳng /staff không qua được kiểm tra. Giao diện ẩn menu theo vai trò nhưng không phải lớp bảo vệ. Bí mật cấu hình ngoài mã nguồn.

Phòng và quy tắc thời gian

Mã	Quy định bắt buộc
BR01	Một cơ sở có 6–10 phòng mẫu. Mỗi phòng có tên, sức chứa, ảnh, tiện nghi, đơn giá giờ và trạng thái hoạt động.
BR02	Giờ mở cửa 09:00–23:00 theo Asia/Ho_Chi_Minh. Mọi thời điểm lưu kiểu timestampz (UTC) trong cơ sở dữ liệu; giao diện hiển thị giờ Việt Nam. Kiểm tra giờ mở cửa được thực hiện tại máy chủ sau khi quy đổi múi giờ.
BR03	Khách đặt từ ngày hiện tại đến hết ngày thứ 14 tính cả hôm nay; bắt đầu cách hiện tại ít nhất 30 phút. Giờ bắt đầu theo mốc :00 hoặc :30.
BR04	Chọn gói 120 hoặc 180 phút. Sau đó dành 30 phút dọn phòng; cả sử dụng và dọn phòng phải kết thúc trước hoặc đúng 23:00.
BR05	Số khách là số nguyên từ 1 đến sức chứa phòng. Mỗi booking chỉ đặt một phòng.
BR06	Phòng không hoạt động không được đặt mới. Không cho đóng phòng khi còn booking xác nhận hoặc đang sử dụng; phải xử lý các booking đó trước.
BR07	Khách đến muộn hoặc về sớm không thay đổi giờ kết thúc hay tiền phòng đã đặt. Chưa có gia hạn trong phiên bản final.
BR08	Đơn giá giờ được chốt vào booking tại thời điểm xác nhận (room_rate_snapshot). Quản lý sửa giá phòng chỉ ảnh hưởng booking tạo sau đó.

Khoảng chiếm dụng và chống trùng

Khoảng chiếm dụng là **[bắt đầu, kết thúc + 30 phút)**, tức nửa mở: chứa thời điểm bắt đầu, không chứa thời điểm kết thúc dọn phòng. Hai booking cùng phòng xung đột khi bắt đầu A < kết thúc chiếm dụng B và bắt đầu B < kết thúc chiếm dụng A. Các khoảng chạm biên được phép.

Ví dụ phòng A được đặt 14:00–16:00: 16:00 chưa đặt tiếp được, 16:30 được phép. Gói 2 giờ muộn nhất bắt đầu 20:30; gói 3 giờ muộn nhất bắt đầu 19:30.

CONFIRMED, IN_USE và COMPLETED giữ khoảng chiếm dụng theo lịch gốc. CANCELLED và NO_SHOW không giữ chỗ. Hoàn thành sớm hoặc kết thúc sớm không giải phóng phòng sớm trong bản này; quy tắc này giữ cho lịch đơn giản và ràng buộc trong cơ sở dữ liệu không đổi.

Cơ chế kỹ thuật bảo đảm quy tắc trên được mô tả ở trang 12: giao dịch có khóa bản ghi phòng cộng với ràng buộc loại trừ trên cột phòng và khoảng thời gian.

Ngoại lệ tại quầy

Nhân viên có thể tạo booking bắt đầu ở mốc 30 phút kế tiếp, không cần khoảng báo trước 30 phút. Vẫn áp dụng sức chứa, giờ đóng cửa và chống trùng. Nếu khách đến sớm, chờ đến giờ bắt đầu để check-in.

Đặt phòng và trạng thái

Dữ liệu khách cần xác nhận

Phòng, ngày và giờ, gói thời lượng, số khách, họ tên và số điện thoại liên hệ; phim tùy chọn; danh sách món tùy chọn; tiền phòng, tiền món và tổng dự kiến. Chính sách hủy và đến muộn phải xuất hiện trước nút xác nhận.

Nguyên tắc tạo booking

- Khách phải đăng nhập trước khi xác nhận. Mỗi tài khoản tối đa 3 booking CONFIRMED có giờ bắt đầu trong tương lai; kiểm tra giới hạn này tại máy chủ, bên trong cùng giao dịch tạo booking.
- Không giữ chỗ trong lúc khách đang chọn phim/món. Khi xác nhận, máy chủ kiểm tra lại phòng, phim, món, giá và thời gian. Giá thay đổi phải được trả về để khách xác nhận lại.
- Tạo booking và các món đặt trước trong một giao dịch: thành công cùng nhau hoặc không lưu gì. Mỗi lần xác nhận có khóa chống gửi lặp (Idempotency-Key); thử lại với cùng khóa trả về kết quả đã lưu, không sinh booking mới.
- Booking hợp lệ được xác nhận tự động, chưa thu tiền. Nếu xung đột, báo phòng đã có người đặt và cho chọn lại; không báo thành công giả.

Trạng thái sử dụng phòng

Trạng thái	Chuyển tiếp hợp lệ và điều kiện
CONFIRMED Đã xác nhận	→ IN_USE: nhân viên check-in bất kỳ lúc nào trước giờ kết thúc (kể cả khách đến muộn; giờ kết thúc không đổi). → CANCELLED: hủy đúng chính sách. → NO_SHOW: nhân viên xác nhận khách chưa đến sau 15 phút. → COMPLETED: đã qua giờ kết thúc mà nhân viên quên check-in nhưng khách thực tế đã dùng phòng; ghi “sử dụng không check-in” theo EX06.
IN_USE Đang sử dụng	→ COMPLETED: check-out bình thường (các món còn tính tiền đã phục vụ và thu đủ), hoặc kết thúc sớm có lý do theo mục xử lý ngoại lệ (trang 8).
COMPLETED Hoàn thành	Trạng thái cuối của việc sử dụng phòng; không sửa món hoặc phim. Trạng thái thanh toán có thể còn UNPAID nếu khách bỏ về.
CANCELLED / NO_SHOW	Trạng thái cuối; không check-in hoặc thanh toán như booking đang dùng.

Trạng thái thanh toán (tách riêng)

Giá trị	Ý nghĩa
UNPAID	Mặc định khi tạo booking; cũng dùng cho trường hợp khách bỏ về chưa trả sau khi phòng đã COMPLETED.
PAID	Đã thu đủ số phải trả (sau điều chỉnh nếu có). Chỉ đặt khi có bản ghi Payment hợp lệ.
WAIVED	Quản lý miễn toàn bộ do sự cố; số phải thu bằng 0, không có Payment, không tính doanh thu.

Hai trạng thái độc lập: khách đã rời phòng không có nghĩa quán đã thu đủ tiền. Báo cáo doanh thu chỉ dựa vào Payment đã ghi nhận.

Chính sách hủy và booking quá hạn

Khách tự hủy nếu còn ít nhất 2 giờ trước giờ bắt đầu. Món đặt trước chưa chuẩn bị được hủy cùng booking; trong bản final chỉ chuẩn bị đồ ăn sau check-in nên hủy trước giờ đến không phát sinh tiền món. Nhân viên có thể hủy CONFIRMED theo yêu cầu khách hoặc sự cố phòng, phải ghi lý do và lịch sử. Không cho hủy IN_USE; dùng kết thúc sớm.

NO_SHOW được đánh dấu thủ công, không tự động, không ghi nhận doanh thu. Khu vực nhân viên có bộ lọc “**Đã qua giờ nhưng chưa xử lý**”: liệt kê booking CONFIRMED có giờ bắt đầu đã qua và booking IN_USE đã quá giờ kết thúc, để nhân viên check-in muộn (nếu chưa tới giờ kết thúc), đánh dấu NO_SHOW, ghi “sử dụng không check-in” hoặc check-out. Hệ thống không tự kết luận thay nhân viên.

Chọn phim trước

Mục đích: Khách chọn phim cho buổi đặt phòng để nhân viên chuẩn bị trước. Việc chọn phim là tùy chọn và đã bao gồm trong giá phòng; khách có thể chọn “Chọn tại quán”.

Mã	Yêu cầu chức năng
MOV01	Danh sách phim có poster, tên, thể loại, thời lượng phút, mô tả và nhãn độ tuổi. Dữ liệu là phim thật: seed 51 phim, nhập thêm tới 5.000 phim từ The Movies Dataset (Kaggle/TMDB) bằng script; API luôn phân trang (24 phim/trang), tìm theo tên, lọc thể loại và lọc “vừa gói”.
MOV02	Chỉ phim ACTIVE mới được chọn. Mỗi booking chọn tối đa một phim. Hệ thống không phát video và không tự khởi động thiết bị.
MOV03	Phim phải dài tối đa thời lượng phòng trừ 10 phút ổn định. Gói 120 phút chọn phim ≤110 phút; gói 180 phút chọn phim ≤170 phút.
MOV04	Nếu phim quá dài, từ chối lựa chọn và gợi ý phim khác hoặc tìm lại gói dài hơn. Không tự kéo dài booking hoặc cộng phí.
MOV05	Khách được đổi phim khi booking CONFIRMED và chưa đến giờ bắt đầu. Phim mới phải còn phục vụ và đủ thời lượng.
MOV06	Đổi phim đặt trạng thái chuẩn bị về PENDING. Nhân viên xem booking sắp đến và đánh dấu READY sau khi chuẩn bị.
MOV07	Nếu phim ngừng phục vụ, booking vẫn còn hiệu lực; đánh dấu UNAVAILABLE, hiển thị cảnh báo cho cả khách và nhân viên để chọn lại.
MOV08	Hai phòng được chọn cùng một phim. Trong mô hình demo, phim không phải tài nguyên đặt độc quyền.

Trạng thái chuẩn bị

NOT_SELECTED: chưa chọn. **PENDING:** đã chọn, chưa chuẩn bị. **READY:** nhân viên đã chuẩn bị. **UNAVAILABLE:** phim đã chọn hiện không thể phục vụ. Đây là trạng thái riêng, không thay thế trạng thái booking.

Chọn phim tại quán

Nhân viên có thể ghi nhận lựa chọn khi check-in, vẫn kiểm tra phim. Vì giờ kết thúc không đổi, thời lượng phim được so với **thời gian còn lại** (giờ kết thúc trừ thời điểm chọn, trừ 10 phút ổn định), không phải thời lượng gói; khách đến muộn chỉ chọn được phim ngắn hơn. Nếu phim chưa sẵn sàng, hiển thị cảnh báo nhưng không tự hủy phòng. Bản demo giả định danh mục do quán chuẩn bị; không tích hợp kho phim bên ngoài.

Khi đánh dấu READY, kiểm tra lựa chọn phim chưa bị thay đổi kể từ lúc nhân viên mở trang bằng cột `movie_version` trên booking (khóa lạc quan); nếu đã thay đổi, tải lại trước khi xác nhận chuẩn bị.

Món, hóa đơn và thanh toán

Đặt món

Khách chọn món khi đặt phòng. Sau khi booking được tạo, món bổ sung được nhân viên nhập tại quầy khi khách đang sử dụng. Mỗi món có tên, giá, ảnh, danh mục và trạng thái đang bán. Số lượng cho mỗi dòng từ 1 đến 20.

Đơn món chuyển **PENDING** → **PREPARING** → **SERVED**. PENDING có thể chuyển CANCELLED. Chỉ bắt đầu chuẩn bị sau check-in; không hủy món đã chuẩn bị trong phạm vi final. Nhân viên xử lý đơn PENDING không còn phục vụ bằng cách hủy và thông báo khách.

Công thức thống nhất

Tiền phòng = Giá giờ đã chốt × Số giờ đặt.

Tiền món = Tổng (giá đã chốt × số lượng) của dòng không bị hủy.

Tổng gốc = Tiền phòng + Tiền món.

Số phải trả = Tổng gốc – Khoản điều chỉnh đã duyệt (mặc định 0).

Dùng số nguyên VND, không dùng số thực cho tiền. Giá hiển thị là giá cuối cùng trong mô hình demo. Không có phí chọn phim, phí dọn phòng riêng, thuế cộng thêm, chiết khấu tự do hoặc gia hạn. Khoản điều chỉnh chỉ phát sinh qua quy trình ngoại lệ ở trang 8 và không vượt quá Tổng gốc.

Khoản	Số tiền
Phòng 14:00–16:00; 100.000đ/giờ	200.000đ
Phim đã chọn	0đ phí bổ sung
2 đồ uống × 35.000đ	70.000đ
Bếp rang gọi thêm	40.000đ
Tổng gốc	310.000đ
Điều chỉnh (máy chiếu lỗi 20 phút; quản lý đã duyệt giảm trước khi thu)	–50.000đ
Số phải trả	260.000đ

Quy trình thu tiền

Nhân viên kiểm tra các món: đơn PENDING phải được phục vụ hoặc hủy, PREPARING phải phục vụ. Máy chủ tính lại hóa đơn từ dữ liệu đã lưu; không tin số tiền do trình duyệt gửi lên. Nhân viên thu tiền, chọn CASH hoặc TRANSFER rồi xác nhận. Chuyển khoản được kiểm tra thủ công ngoài hệ thống.

Số thu bằng Số phải trả; chỉ trừ điều chỉnh khi điều chỉnh đó đã **APPROVED**. Nếu điều chỉnh còn chờ duyệt, nhân viên hoặc thu đủ Tổng gốc (khách đồng ý) hoặc để booking UNPAID chờ quản lý quyết định; không thu theo mức giảm dự kiến. Tạo bản ghi Payment, đặt payment_status = PAID và chuyển booking sang COMPLETED trong cùng giao dịch. Mỗi booking có tối đa một Payment; thử lại với cùng Idempotency-Key trả lại kết quả cũ, khóa khác trên booking đã PAID trả 409. Không cho thêm món, sửa giá hoặc sửa điều chỉnh sau khi đã thu.

Doanh thu và giới hạn

Doanh thu là tổng Payment đã ghi nhận, nhóm theo ngày thanh toán (paid_at). Booking CONFIRMED, CANCELLED, NO_SHOW, WAIVED và COMPLETED còn UNPAID không tính doanh thu; hai loại sau xuất hiện trong báo cáo riêng “Chưa thu và miễn” để quản lý đối chiếu. Chưa hỗ trợ chia hóa đơn, thanh toán từng phần, hoàn tiền hoặc tự xác thực ngân hàng.

Xử lý ngoại lệ tại quầy

Mục này bổ sung để booking không bị kẹt ở IN_USE và để tiền được ghi nhận đúng khi có sự cố. Nguyên tắc: nhân viên ghi nhận và tiếp tục phục vụ; mọi khoản giảm hoặc miễn phải được quản lý duyệt **trước khi** áp dụng vào sổ thu; mọi thao tác có lý do và lịch sử.

Mã	Tình huống	Cách xử lý bắt buộc
EX01	Kết thúc sớm có lý do	Nhân viên chọn “Kết thúc sớm”, nhập lý do (sự cố thiết bị, khách yêu cầu, khách rời đi, khác). Booking IN_USE → COMPLETED, lưu ended_early_reason và thời điểm. Không đổi giờ kết thúc, không giải phóng phòng sớm. Món xử lý theo EX02; tiền theo EX03–EX05.
EX02	Món còn dở khi kết thúc sớm	PENDING → CANCELLED, không tính tiền. SERVED → giữ nguyên, tính tiền. PREPARING → tính tiền (nguyên liệu đã dùng); nếu quán muốn không thu vì sự cố, đưa vào Điều chỉnh chờ quản lý duyệt. Không đánh dấu “đã phục vụ” chỉ để vượt điều kiện thanh toán.
EX03	Sự cố, quán chấp nhận giảm tiền	Nhân viên tạo Điều chỉnh (số tiền giảm, lý do) ở trạng thái PENDING_APPROVAL. Quản lý duyệt hoặc từ chối trước khi thu tiền; quản lý có thể duyệt từ xa bằng tài khoản của mình trên điện thoại. Chỉ sau APPROVED nhân viên mới thu. Số phải trả đã giảm. Nếu khách không thể chờ: thu đủ Tổng gốc (khách đồng ý) hoặc kết thúc phòng và để UNPAID (EX05); không thu theo mức giảm chưa duyệt.
EX04	Quán miễn toàn bộ do sự cố	Chỉ quản lý được miễn, bằng cách duyệt một Điều chỉnh loại WAIVE. Số phải trả bằng 0; payment_status = WAIVED; lưu lý do và người duyệt; không tạo Payment; không ghi thành đã thu. Nhân viên chỉ được tạo đề nghị; khi chưa duyệt, booking giữ UNPAID.
EX05	Khách rời đi chưa trả tiền hoặc chờ duyệt	Kết thúc sử dụng phòng (EX01). Booking COMPLETED nhưng payment_status = UNPAID, có trong báo cáo “Chưa thu và miễn”, không tính doanh thu. Khi quản lý duyệt xong hoặc khách quay lại trả, ghi Payment bình thường với ngày thực thu. Chưa làm công nợ đầy đủ.
EX06	Booking quá hạn chưa xử lý	Bộ lọc “Đã qua giờ nhưng chưa xử lý”. CONFIRMED chưa tới giờ kết thúc: check-in muộn hoặc NO_SHOW. CONFIRMED đã qua giờ kết thúc: NO_SHOW, hoặc “sử dụng không check-in” → COMPLETED kèm ghi chú rời thu tiền như bình thường nếu khách thực tế đã dùng phòng. IN_USE đã qua giờ kết thúc: check-out hoặc EX05. Không có job tự động kết luận.
EX07	Sự cố phòng trước giờ đến	Nhân viên hủy CONFIRMED với lý do sự cố, hoặc tạo booking mới ở phòng khác cho khách rồi hủy booking cũ; cả hai thao tác đi qua cơ chế chống trùng bình thường. Chưa hỗ trợ đổi phòng trong một thao tác.

Quy tắc chung cho điều chỉnh

- Điều chỉnh chỉ có hiệu lực khi APPROVED và luôn xảy ra **trước** khi thu tiền. Mỗi booking có tối đa một điều chỉnh đang hiệu lực; thay đổi trước khi duyệt tạo bản ghi mới và vô hiệu bản cũ, giữ lịch sử.
- Số tiền điều chỉnh là số nguyên VND, lớn hơn 0 và không vượt quá Tổng gốc (WAIVE bằng đúng Tổng gốc). Hóa đơn luôn hiển thị Tổng gốc, Điều chỉnh và Số phải trả.
- Không thu tiền hoặc miễn tiền cho booking CANCELLED, NO_SHOW. Sau khi booking đã PAID không tạo, sửa hay duyệt điều chỉnh nữa; không có chức năng mở lại hóa đơn hay hoàn tiền trong bản này.
- Mọi hành động trong mục này ghi vào lịch sử với người thực hiện, thời điểm và lý do; báo cáo “Chưa thu và miễn” liệt kê đầy đủ để quản lý xử lý hàng đợi duyệt và kiểm tra cuối ngày.

Giới hạn đã chấp nhận: không công nợ, không hoàn tiền, không thanh toán từng phần, không mở lại hóa đơn. Nguyên tắc “duyet trước, thu sau” giữ cho mỗi booking chỉ có một Payment và không bao giờ phát sinh khoản thiếu sau khi đã PAID.

Các luồng sử dụng chi tiết

Use case	Tiền điều kiện và luồng chính	Ngoại lệ / kết quả
UC01 Tìm và đặt phòng	Khách chọn ngày, giờ, gói, số người → xem phòng phù hợp → đăng nhập → chọn phim/món → xem giá → xác nhận với Idempotency-Key.	Không phòng: đổi bộ lọc. Giá/món/phim thay đổi: xác nhận lại. Xung đột: 409, gợi ý khung giờ khác. Thành công: mã booking, chi tiết và lịch sử.
UC02 Xem, hủy, đổi phim	Khách mở booking của mình → thao tác được phép → máy chủ kiểm tra trạng thái và thời hạn → lưu.	Không đủ quyền hoặc quá hạn: từ chối. Hủy hợp lệ giải phóng lịch; đổi phim đặt lại PENDING.
UC03 Chuẩn bị và check-in	Nhân viên mở lịch ngày → xem phim cần chuẩn bị → đánh dấu READY → tìm mã hoặc số điện thoại → kiểm tra khách và thời gian → check-in.	Booking đã hủy/hoàn thành/không đến: không check-in. Trễ 15 phút hiện nhãn cảnh báo; nhân viên quyết định NO_SHOW.
UC04 Khách tại quầy	Nhân viên chọn phòng và giờ còn trống → nhập tên, số điện thoại, số khách → chọn phim/món → tạo booking.	Có thể không có tài khoản khách; không tự tạo tài khoản. Áp dụng cùng cơ chế chống trùng.
UC05 Phục vụ và thanh toán	IN_USE → thêm món → chuẩn bị/phục vụ → kiểm tra hóa đơn (Tổng gốc, Điều chỉnh, Số phải trả) → thu tiền → ghi nhận → COMPLETED + PAID.	Món chưa xử lý: chặn kết thúc. Gửi lặp thanh toán: trả kết quả cũ. Số tiền do trình duyệt sửa: không chấp nhận.
UC06 Ngoại lệ tại quầy	Nhân viên chọn “Kết thúc sớm” (món dở xử lý theo EX02) hoặc tạo “Điều chỉnh” với lý do, số tiền → quản lý duyệt/từ chối (tại chỗ hoặc từ xa) → nhân viên thu Số phải trả sau duyệt, hoặc để UNPAID nếu khách rời trước khi duyệt.	Điều chỉnh vượt Tổng gốc: từ chối. Chưa duyệt: không thu theo mức giảm. Quản lý từ chối: thu đủ Tổng gốc khi khách trả. Miễn toàn bộ chỉ quản lý làm được.
UC07 Quản lý danh mục	Quản lý tạo/sửa phòng, phim, món; cập nhật giá, tình trạng; tạo/khóa nhân viên; xem báo cáo.	Không xóa cứng dữ liệu được booking tham chiếu. Giá mới áp dụng cho lần đặt sau; lịch sử giữ nguyên. Khóa nhân viên thu hồi phiên ngay.

Đường đi khi có lỗi

Khi lỗi mạng sau xác nhận, giao diện cho thử lại với cùng khóa thao tác để nhận kết quả đã lưu. Không yêu cầu khách tạo một booking khác ngay. Lỗi kiểm tra dữ liệu chỉ rõ trường cần sửa; lỗi hệ thống có mã tra cứu, không hiển thị thông tin nội bộ. Phản hồi 409 do trùng phòng được hiển thị là “Khung giờ vừa được đặt”, kèm gợi ý khung giờ gần nhất còn trống của cùng phòng.

Màn hình và trải nghiệm

Khu vực	Màn hình và nội dung cần có
Khách hàng	Trang chủ có form tìm phòng; kết quả có giá và sức chứa; chi tiết phòng có ảnh, tiện nghi và chính sách.
Quy trình đặt	Bước 1 phòng/giờ → bước 2 phim hoặc bỏ qua → bước 3 món → bước 4 kiểm tra và xác nhận. Luôn thấy tóm tắt phòng, thời gian và tổng dự kiến.
Tài khoản khách	Đăng ký, đăng nhập, đăng xuất; lịch sử lọc sắp tới/đã kết thúc; chi tiết có mã, trạng thái, phim, món, tiền và nút hủy/đổi phim khi được phép.
Nhân viên	Lịch ngày dạng bảng thời gian theo phòng; màu trạng thái kèm chữ; tìm booking theo mã/số điện thoại; bộ lọc “Đã qua giờ nhưng chưa xử lý”; trang chi tiết chứa thao tác theo trạng thái (check-in, NO_SHOW, kết thúc sớm, hủy).
Phục vụ	Danh sách đơn món theo trạng thái; danh sách phim cần chuẩn bị; hóa đơn tách tiền phòng, tiền món, Tổng gốc, Điều chỉnh, Số phải trả; ghi phương thức thu tiền; nút “Điều chỉnh” và “Chưa thu”.
Quản lý	CRUD phòng, phim, menu; tạo/khóa nhân viên; báo cáo theo khoảng ngày; hàng đợi duyệt điều chỉnh; báo cáo “Chưa thu và miễn”.

Báo cáo bắt buộc

- Số booking theo ngày bắt đầu và trạng thái sử dụng; không trộn ngày tạo với ngày sử dụng.
- Doanh thu đã thu theo ngày `Payment.paid_at`. Hiển thị tổng và bảng chi tiết để đối chiếu.
- Giờ phòng đã sử dụng: tổng thời lượng đặt của booking COMPLETED, nhóm theo ngày bắt đầu. Chỉ số này không phải thời lượng hiện diện thực tế.
- Chưa thu và miễn: danh sách booking COMPLETED còn UNPAID, WAIVED, và điều chỉnh theo trạng thái duyệt, kèm lý do và người thực hiện.

Yêu cầu giao diện chung

Responsive ở độ rộng 360px và màn hình máy tính. Form có nhãn, báo lỗi cạnh trường và giữ dữ liệu khi lỗi. Mỗi trang có trạng thái tải, rỗng, lỗi và thành công. Trạng thái không chỉ phân biệt bằng màu; thao tác chính dùng được bằng bàn phím.

Hiển thị tiền dạng 100.000đ, ngày theo dd/mm/yyyy và giờ 24h. Có xác nhận trước khi hủy booking, kết thúc sớm và ghi nhận thu tiền. Không hiển thị ID kỹ thuật hoặc thông báo cơ sở dữ liệu cho người dùng.

Mô hình dữ liệu PostgreSQL

Bảng	Trường chính / ghi chú
user	id, name, email (unique), phone, password_hash, role (CUSTOMER/STAFF/MANAGER), is_active, created_at.
session	Schema mặc định của connect-pg-simple: sid (PK), sess (json), expire; thêm cột sinh user_id int GENERATED ALWAYS AS ((sess->>'userId')::int) STORED kèm chỉ mục để khóa tài khoản xóa mọi phiên theo user_id (Phụ lục A).
login_attempt	id, email, ip, succeeded, attempted_at; phục vụ giới hạn thử đăng nhập; dọn định kỳ.
room	id, name, capacity, description, amenities (jsonb), hourly_price_vnd, is_active; room_image lưu URL và thứ tự ảnh.
movie	id, title, genre, duration_minutes, age_label, description, poster_url, is_active.
booking	id, code (unique), customer_id (nullable với khách quầy), created_by, source (ONLINE/COUNTER), room_id, contact_name, contact_phone, guest_count, start_at, end_at, occupied_until, status , payment_status , room_rate_snapshot, room_total, ended_early_reason, ended_at, created_at.
booking (phim)	movie_id (nullable), movie_title_snapshot, movie_duration_snapshot, preparation_status, movie_version. Lưu ảnh chụp tên/thời lượng để lịch sử không đổi.
menu_item	id, name, category, image_url, price_vnd, is_active.
food_order	id, booking_id, status, created_by, created_at; một booking có nhiều đơn khi gọi thêm.
food_order_item	id, order_id, menu_item_id, item_name_snapshot, unit_price_vnd, quantity (1–20).
adjustment	id, booking_id, amount_vnd (>0, ≤ tổng gốc), reason, kind (REDUCE/WAIVE), status (PENDING_APPROVAL/APPROVED/REJECTED), created_by, approved_by, decided_at, is_current. Mỗi booking tối đa một dòng is_current = true (unique partial index). Chỉ APPROVED mới được trừ vào Số phải trả; không tạo/sửa sau khi booking PAID.
payment	id, booking_id (unique), original_total_vnd, adjustment_vnd, amount_vnd, method (CASH/TRANSFER), paid_at, recorded_by, idempotency_key (unique).
booking_status_history	id, booking_id, field (status/payment_status/preparation), old_value, new_value, actor_id, changed_at, reason.
idempotency_request	actor_id, key, request_hash, status (IN_PROGRESS/DONE), response_code, response_body, created_at; unique (actor_id, key); chống tạo booking và thanh toán lặp; được ghi ngoài giao dịch nghiệp vụ (trang 12).

Quan hệ và ràng buộc

user 1-N booking; room 1-N booking; movie 1-N booking; booking 1-N food_order; food_order 1-N food_order_item; booking 1-N adjustment; booking 0-1 payment. Mọi tham chiếu dùng khóa ngoại. Không xóa cứng phòng, phim, món đã phát sinh giao dịch; dùng is_active.

Ràng buộc CHECK: guest_count từ 1 đến capacity (kiểm tra ở tầng ứng dụng vì liên bảng), quantity 1–20, giá và tiền không âm, start_at < end_at, end_at <= occupied_until, status và payment_status thuộc tập giá trị cho phép. Ràng buộc loại trừ chống giao nhau trên (room_id, khoảng chiếm dụng) cho status CONFIRMED/IN_USE/COMPLETED được mô tả ở trang 12 và Phụ lục A.

Chỉ mục

booking (room_id, start_at); booking (customer_id, start_at desc); booking (status, start_at) cho bộ lọc quá hạn; payment (paid_at); session (expires_at); login_attempt (email, attempted_at); adjustment (status). Chỉ mục GiST do ràng buộc loại trừ tự tạo phục vụ luôn truy vấn tìm phòng trống.

Chống đặt trùng và giao dịch

Đây là yêu cầu P0 và là nội dung kỹ thuật chính khi Q&A. Nhóm dùng hai lớp bảo vệ độc lập: lớp ứng dụng để trả về thông báo rõ ràng, lớp cơ sở dữ liệu để bảo đảm đúng đắn ngay cả khi mã ứng dụng có lỗi hoặc có nhiều tiến trình máy chủ.

Lớp 1 – Giao dịch có khóa bản ghi phòng

Bước	Thao tác trong một giao dịch (mức cô lập READ COMMITTED là đủ nhờ khóa)
1	Trước giao dịch chính , trong một giao dịch ngắn riêng: INSERT ... ON CONFLICT (actor_id, key) DO NOTHING RETURNING id. Không có dòng trả về nghĩa là khóa đã tồn tại: cùng request_hash và đã DONE → trả lại phản hồi cũ; đang IN_PROGRESS → 409 “đang xử lý”; khác request_hash → 422. Không bắt lỗi trùng khóa rồi tiếp tục dùng giao dịch đó, vì PostgreSQL hủy giao dịch sau lỗi (25P02).
2	SELECT ... FROM room WHERE id = \$1 FOR UPDATE. Các yêu cầu khác cùng phòng xếp hàng tại đây; khác phòng không bị chặn. Từ chối nếu phòng không hoạt động.
3	Kiểm tra quy tắc thời gian (BR02–BR04), sức chứa, phim (MOV02–MOV03), món còn bán; tính tiền từ giá trong cơ sở dữ liệu.
4	Truy vấn booking giao nhau trên phòng với status CONFIRMED/IN_USE/COMPLETED. Nếu có, trả 409 kèm khung giờ trống gần nhất.
5	Với khách online: SELECT ... FROM "user" WHERE id = \$2 FOR UPDATE rồi đếm booking CONFIRMED tương lai; quá 3 trả 422.
6	INSERT booking, food_order, food_order_item, booking_status_history. COMMIT. Sau COMMIT, cập nhật idempotency_request thành DONE kèm phản hồi; nếu giao dịch thất bại (409/422/lỗi), xóa dòng khóa để khách thử lại với cùng khóa.

Lớp 2 – Ràng buộc loại trừ trong PostgreSQL

Bảng booking có ràng buộc EXCLUDE USING gist (room_id WITH =, tstzrange(start_at, occupied_until, '[')') WITH &&) WHERE (status IN ('CONFIRMED', 'IN_USE', 'COMPLETED')) (cần extension btree_gist). Hai dòng cùng phòng có khoảng chiếm dụng giao nhau không thể cùng tồn tại; PostgreSQL trả lỗi 23P01 exclusion_violation và giao dịch bị hủy. Ứng dụng ánh xạ lỗi này thành 409 giống bước 4. Khoảng nửa mở [) làm cho các booking chạm biên (16:30 sau 14:00–16:00) hợp lệ, đúng quy tắc trang 4.

Hủy hoặc NO_SHOW đổi status ra khỏi tập trong mệnh đề WHERE nên tự động không còn giữ chỗ, không cần xóa dòng. Kết thúc sớm giữ nguyên occupied_until nên không ảnh hưởng ràng buộc.

Áp dụng với Prisma

- Dùng giao dịch tương tác prisma.\$transaction(async (tx) => { ... }); hai câu khóa FOR UPDATE chạy bằng tx.\$queryRaw, các thao tác còn lại dùng API Prisma trong cùng tx.
- Prisma schema không biểu diễn được EXCLUDE; tạo migration bằng prisma migrate dev --create-only rồi thêm SQL ràng buộc vào file migration (Phụ lục A). Kiểm tra bằng cách bật btree_gist trong cùng migration.
- Bắt lỗi trong khối giao dịch: mã lỗi PostgreSQL 23P01 (loại trừ) và 23505 (trùng khóa duy nhất, dùng cho idempotency và payment) được ánh xạ thành 409; các lỗi khác thành 500 có mã tra cứu.
- Đặt statement_timeout khoảng 5 giây cho giao dịch tạo booking để một yêu cầu treo không giữ khóa phòng vô hạn.

Thanh toán và điều chỉnh

Ghi nhận thu tiền dùng cùng khuôn mẫu: xử lý Idempotency-Key trước; trong giao dịch khóa dòng booking FOR UPDATE, kiểm tra trạng thái món, chỉ trừ điều chỉnh APPROVED, tính lại hóa đơn, chèn payment (unique booking_id), cập nhật status/payment_status và lịch sử, COMMIT. Thử lại cùng khóa trả kết quả cũ; khóa khác trên booking đã PAID trả 409. Kết thúc sớm, tạo và duyệt điều chỉnh cũng khóa dòng booking để hai người không thao tác chéo.

Kiểm chứng: T02 và T14 (trang 14) chạy 200 yêu cầu thật đồng thời; sau khi chạy, câu truy vấn đếm booking giữ chỗ trên phòng/giờ đó phải trả về đúng 1. Kịch bản PF02 (trang 15) lặp lại phép thử này dưới công cụ đo tải.

API và nguyên tắc triển khai

Danh sách dưới là hợp đồng API khởi đầu để nhóm chia việc. Tên đường dẫn có thể điều chỉnh nhưng không thay đổi quy tắc nghiệp vụ. Tiền tố /api; phản hồi JSON thống nhất { ok, data, error }.

Phương thức / đường dẫn	Mục đích / quyền
POST /auth/register, /auth/login POST /auth/logout	Đăng ký khách, đăng nhập (tạo phiên, cookie), đăng xuất (xóa phiên). Đăng nhập có giới hạn thử.
GET /me	Thông tin và vai trò của phiên hiện tại; 401 nếu phiên không còn hoặc tài khoản bị khóa.
GET /rooms/availability	Tra phòng bằng date, startTime, duration, guests; công khai. Đây là API được đo ở PF01.
GET /rooms/:id GET /movies; GET /menu-items	Chi tiết phòng và danh mục còn phục vụ; công khai.
POST /bookings	Khách đăng nhập tạo booking; bắt buộc header Idempotency-Key; 409 khi trùng, 422 khi vi phạm quy tắc.
GET /me/bookings GET /bookings/:id	Danh sách cá nhân (phân trang); chi tiết kiểm tra chủ sở hữu hoặc quyền nội bộ.
POST /bookings/:id/cancel PATCH /bookings/:id/movie	Hủy hoặc đổi phim; kiểm tra thời hạn, trạng thái và movie_version.
GET /staff/bookings?date= GET /staff/bookings/overdue	Lịch toàn cơ sở theo ngày; danh sách “đã qua giờ nhưng chưa xử lý”.
POST /staff/bookings	Tạo booking tại quầy; cùng cơ chế chống trùng và Idempotency-Key.
POST /staff/bookings/:id/check-in POST /staff/bookings/:id/no-show	Chuyển trạng thái hợp lệ; ghi lịch sử.
PATCH /staff/bookings/:id/preparation POST /staff/bookings/:id/orders PATCH /staff/orders/:id/status	Chuẩn bị phim; thêm món cho booking IN_USE; cập nhật phục vụ.
POST /staff/bookings/:id/checkout	Thu tiền và hoàn thành nguyên tử; Idempotency-Key bắt buộc; cùng khóa trả kết quả cũ, khóa khác trên booking đã PAID trả 409.
POST /staff/bookings/:id/end-early POST /staff/bookings/:id/adjustments POST /staff/bookings/:id/mark-used	Kết thúc sớm có lý do, kèm cách xử lý món dở (EX01–EX02); tạo điều chỉnh chờ duyệt (EX03); ghi “sử dụng không check-in” (EX06).
POST /admin/adjustments/:id/approve reject GET /admin/adjustments?status=PENDING_APPROVAL	Quản lý duyệt/từ chối điều chỉnh (REDUCE hoặc WAIVE, EX03–EX04); hàng đợi chờ duyệt dùng được trên điện thoại.
/admin/rooms, /movies, /menu-items /admin/staff, GET /admin/reports/*	Tạo/sửa/ngừng phục vụ; quản lý nhân viên (khóa xóa phiên); báo cáo: bookings, revenue, room-hours, unpaid-waived.

Phản hồi nhất quán và an toàn

401 chưa đăng nhập hoặc phiên hết hiệu lực, 403 không có quyền, 404 không tìm thấy, 409 xung đột lịch/trạng thái/thao tác lặp, 422 dữ liệu không hợp lệ hoặc vi phạm quy tắc nghiệp vụ, 429 quá giới hạn thử đăng nhập. Không tin giá, vai trò hoặc trạng thái tùy ý từ trình duyệt. Mọi dữ liệu vào được kiểm tra bằng schema; truy vấn qua Prisma hoặc tham số hóa để tránh SQL injection; header bảo mật bằng helmet; HTTPS trên bản triển khai.

Chất lượng và nghiệm thu

Mã	Tình huống	Kết quả mong đợi
T01	Đặt hợp lệ, chọn phim/món	Lưu đúng một booking, đúng giá và chi tiết.
T02	200 yêu cầu từ 200 khách hợp lệ khác nhau, cùng phòng/giờ, gửi đồng thời	Đúng một booking thành công; còn lại 409; không lỗi 500; đếm trong cơ sở dữ liệu bằng 1.
T03	Đặt giao thời gian dọn; đặt chạm biên	Giao nhau bị chặn; bắt đầu đúng occupied_until được phép.
T04	Quá sức chứa, quá khứ, quá 23:00	Từ chối và chỉ rõ nguyên nhân.
T05	Khách truy cập booking người khác / API nội bộ	Không lộ dữ liệu, không thực hiện thao tác.
T06	Hủy trước đúng 2 giờ; hủy muộn hơn	Đúng 2 giờ được hủy; muộn hơn bị chặn với khách.
T07	Phim 110/111 phút cho gói 120 phút	110 được chọn, 111 bị chặn.
T08	Đổi phim sau READY; phim bị ngừng	Đổi về PENDING; phim ngừng hiện UNAVAILABLE.
T09	Giá thay đổi sau đặt; sửa tổng tiền phía khách	Booking cũ giữ giá; máy chủ tính đúng, không tin tổng gửi lên.
T10	Bấm đặt/thu tiền lặp, lỗi mạng rồi thử lại	Không sinh booking hoặc payment trùng; nhận lại kết quả cũ.
T11	Thanh toán khi món đang chuẩn bị	Chặn; khi đã phục vụ/hủy hợp lệ mới cho hoàn thành.
T12	Báo cáo có booking hủy, no-show, chưa thu, miễn	Chỉ Payment đã thu vào doanh thu; chưa thu và miễn xuất hiện ở báo cáo riêng.
T13	Hai booking cùng phim, khác phòng	Cả hai thành công nếu phòng đủ điều kiện.
T14	Đặt tại quầy cạnh tranh với đặt online	Dùng cùng cơ chế chống trùng, không giữ hai booking trùng.
T15	Kết thúc sớm do sự cố khi có món PENDING, PREPARING, SERVED; nhân viên đề nghị giảm 50.000đ; thử thu trước khi duyệt; quản lý duyệt; thu	PENDING bị hủy không tính tiền, PREPARING và SERVED tính tiền; thu khi điều chỉnh còn PENDING_APPROVAL bị chặn (chỉ cho thu đủ Tổng gốc hoặc để UNPAID); sau APPROVED, Payment ghi original 310.000đ, adjustment 50.000đ, amount 260.000đ; booking COMPLETED + PAID.
T16	Khách bỏ về chưa trả; quản lý miễn một booking khác	Booking 1: COMPLETED + UNPAID, không có trong doanh thu. Booking 2: WAIVED, không Payment, không doanh thu. Cả hai xuất hiện ở “Chưa thu và miễn”.
T17	Khóa tài khoản nhân viên đang đăng nhập	Yêu cầu kế tiếp từ phiên đó trả 401 ngay; không cần chờ hết hạn cookie.
T18	Đăng nhập sai 6 lần trong 10 phút	Lần thứ 6 trả 429; sau 10 phút đăng nhập đúng thành công.

Kiểm thử T02 và T14 chạy với kết nối đồng thời thật (không tuần tự) trên môi trường Docker của nhóm, có ghi lại lệnh chạy và kết quả. Kiểm thử phân quyền, dữ liệu đầu vào và truy vấn an toàn được viết bằng Vitest + Supertest và chạy trong CI cho mỗi pull request.

Kiểm thử hiệu năng và tải

Phần nâng cao có bằng chứng. Mọi con số dưới đây là **mục tiêu**, không phải kết quả đã đạt; kết quả thực nghiệm bổ sung vào bảng khi có, kèm ngày đo và cấu hình. Chỉ tối ưu khi số đo cho thấy cần; không cố tạo bản đầu chậm để làm đẹp số liệu.

Môi trường và công cụ

- Môi trường đo cố định: một máy chạy Docker Compose (API, PostgreSQL), một máy chạy Locust cùng mạng LAN; ghi CPU, RAM, hệ điều hành, phiên bản Node và PostgreSQL; ghi rõ đo qua LAN, không phải Internet.
- Công cụ chính: **Locust** (Python) cho cả ba kịch bản; kịch bản mẫu ở Phụ lục B. Mỗi phép thử chỉ định rõ lớp người dùng (Customer hoặc Contender) trên dòng lệnh, vì Locust mặc định chạy mọi lớp có trong file. Không dùng thêm công cụ tải khác nếu không có mục đích rõ.
- Dữ liệu: 10 phòng, 30 phim, 20 món, 300 tài khoản, 10.000 booking rải trong **12 tháng lịch sử** (65% COMPLETED, 25% CANCELLED, 10% NO_SHOW; chỉ COMPLETED giữ chỗ, tương đương ~36% của trần 5 lượt/phòng/ngày = 18.250 lượt/năm). 14 ngày tới lấp ~50% để PF01/PF03 còn phòng; phòng 1 lúc 19:00 ngày PF_DATE để trống cho PF02. Script Python có seed cố định; khôi phục trước mỗi lần đo.
- Độ trễ lấy từ Locust phía client; tài nguyên máy chủ từ docker stats; truy vấn chậm từ pg_stat_statements.

Ba kịch bản

Mã	Kịch bản và tham số tải	Cần chứng minh và cách kết luận
PF01	Tìm phòng: GET /rooms/availability, tham số ngẫu nhiên trong 14 ngày; 50 người dùng ảo, spawn rate 10/giây, wait time 2–5 giây, duy trì 5 phút.	p50/p95 độ trễ, yêu cầu/giây, tỷ lệ lỗi (chỉ 5xx và timeout). Mục tiêu đề xuất cho riêng API này ở tải trên: p95 < 300 ms, lỗi < 1%. Đo trước/sau khi thêm chỉ mục hoặc sửa truy vấn trong cùng điều kiện; nếu bản đầu đã đạt thì ghi nhận, không tối ưu thêm.
PF02	Tranh phòng: 200 người dùng ảo, mỗi người một tài khoản và Idempotency-Key riêng; đăng nhập xong chờ đủ 200 người rồi cùng gửi POST /bookings cho một phòng/giờ hợp lệ trong 14 ngày tới; mỗi người gửi đúng một lần.	Đúng 1 phản hồi 201, 199 phản hồi 409, 0 phản hồi 5xx/422; câu đếm trong CSDL bằng 1. 409 là kết quả đúng, cấu hình là thành công trong Locust ; 422 trong PF02 là dữ liệu thử sai, tính là thất bại. Ghi p95 để thấy chi phí xếp hàng khóa.
PF03	Tăng dần: luồng tìm phòng → xem một phòng trong kết quả → đặt đúng phòng đó (10% lượt), wait time 2–5 giây; tăng 0 → 100 → 200 → 300 trong 3 phút, giữ 300 trong 3 phút, giảm về 50 trong 2 phút. 422 do quá 3 booking tính là phản hồi đúng.	Khám phá giới hạn: mức tải mà p95 hoặc tỷ lệ lỗi tăng mạnh; có phục hồi khi giảm tải không. Không cam kết độ trễ ở tải cao nhất; báo cáo “điểm suy giảm” và nguyên nhân quan sát được (CPU, kết nối CSDL, khóa).

Bảng kết quả (điền khi đo)

Kịch bản	Ngày đo	Phiên bản mã	p50	p95	RPS	Lỗi hệ thống	Kết luận
PF01 – trước tối ưu	chưa đo						
PF01 – sau tối ưu	chưa đo						
PF02 – tranh phòng	chưa đo						201 = ? / 409 = ? / DB = ?
PF03 – tăng dần	chưa đo						điểm suy giảm = ? VU

Tối ưu được phép; giao diện

Xuất phát từ số đo: thêm/sửa chỉ mục, sửa truy vấn N+1, phân trang danh sách, điều chỉnh pool kết nối. Cache tìm phòng chỉ thêm khi PF01 không đạt, ghi rõ TTL; bước xác nhận luôn kiểm tra trực tiếp trong giao dịch. Giao diện: Lighthouse trên trang chủ, kết quả tìm phòng và chi tiết phòng ở chế độ di động, mỗi trang 5 lần cùng điều kiện, báo cáo trung vị; là bằng chứng bổ sung, không thay thế đo tải máy chủ.

Phân công và tiến độ

Người	Thành viên	Phạm vi sở hữu	Phần phối hợp
1	Dương	Tài khoản và tích hợp: đăng nhập, session, phân quyền, quản lý nhân viên; cấu trúc dự án, CI, triển khai	Dựng bố cục chung; điều phối tích hợp và xử lý lỗi môi trường; giữ quyền merge schema/migration
2	Chúc	Phòng và tìm phòng: danh mục/chi tiết phòng, quản lý phòng và giá, tìm phòng trống	Cùng người 3 thống nhất truy vấn thời gian; tối ưu tìm phòng
3	Hải Anh	Booking: tạo/hủy booking, chống trùng, chống gửi lặp, lịch nhân viên, check-in, NO_SHOW	Cùng người 6 thống nhất vòng đời booking và khóa giao dịch; chủ trì Q&A kỹ thuật
4	Thành Lê	Phim và hiệu năng: danh mục, chọn/đổi phim, kiểm tra thời lượng, chuẩn bị phim	Chủ trì Locust, dữ liệu benchmark và báo cáo hiệu năng
5	Sơn	Menu và đơn món: quản lý menu, đặt trước/gọi thêm, trạng thái phục vụ, xử lý món dở	Cung cấp tiền món và trạng thái hợp lệ cho thanh toán
6	Công Thành	Thanh toán, ngoại lệ và báo cáo: hóa đơn, thu tiền, kết thúc sớm, đề nghị/duyet giảm-miễn, báo cáo	Cùng người 3 và 5 hoàn thiện luồng kết thúc phòng

Mỗi người làm **giao diện, API, dữ liệu mẫu và kiểm thử phần mình**. Người 1 điều phối tích hợp, nhưng người gây ra lỗi ở phần nào vẫn chịu trách nhiệm sửa phần đó. **Hợp đồng ranh giới chốt ngày 1:** người 3 cung cấp hàm chuyển trạng thái `transitionBooking(id, to, actor, reason)` có khóa dòng và ghi lịch sử, cùng hàm kiểm tra giao nhau dùng chung với người 2; người 5 cung cấp hàm trả về tiền món và trạng thái món hợp lệ; người 6 chỉ gọi hai hàm đó, không tự sửa status hay tính lại tiền món. Mỗi thành viên dùng AI hỗ trợ theo đúng trang đặc tả của module mình; mã sinh ra phải kèm kiểm thử theo mã T ở trang 14.

Ngày	Đầu ra bắt buộc (lịch 1 tuần, giữ nguyên toàn bộ phạm vi)
1	Bộ nền dùng chung chạy được: monorepo, Docker Compose PostgreSQL, <code>schema.prisma</code> đủ mọi bảng trang 11, migration EXCLUDE và cột sinh session, seed cơ bản, khung Express và khung giao diện nhiều trang (Vite, shared/, một trang mẫu cho mỗi module). Mỗi người mở module bằng dữ liệu giả. Người 3: T02 tự động xanh. Cuối ngày công bố ba hàm ranh giới và hợp đồng API.
2	Auth/session/giới hạn thử (1); API bookings, booking của tôi, hủy (3); tìm phòng và quản lý phòng (2); chuẩn bị/đổi phim (4); trạng thái món và món dở (5); checkout, điều chỉnh, duyệt (6).
3	Tích hợp lần 1: tìm → đặt → phim/món → xem → hủy; lịch nhân viên, check-in, NO_SHOW; chống trùng ở cả hai lớp; T01–T10 chạy được.
4	Phục vụ → thu tiền → kết thúc sớm → điều chỉnh → báo cáo; T11–T18; người 1 triển khai bản đầu để phát hiện lỗi môi trường.
5	Người 4 chạy bộ sinh dữ liệu và PF01–PF03, tối ưu có căn cứ rồi đo lại, Lighthouse; cả nhóm rà giao diện responsive và trạng thái tải/rỗng/lỗi.
6	Đóng băng tính năng đầu ngày. Chỉ sửa lỗi; người 1 chạy đủ T01–T18 trên bản triển khai; slide, video dự phòng, hướng dẫn cài đặt; tập trình bày bốc thăm lượt 1.
7	Dự phòng cho lỗi còn sót; tập trình bày lượt 2; chốt bản triển khai và hồ sơ bàn giao.

Hồ sơ bàn giao và điều kiện kết thúc

Mã nguồn và hướng dẫn cài đặt; cấu hình mẫu không có bí mật; migration và seed; ERD; tài liệu API; tài khoản demo ba vai trò; báo cáo kiểm thử T01–T18 kèm kết quả thực; báo cáo hiệu năng PF01–PF03 kèm lệnh chạy và ảnh chụp; URL demo; slide 5–7 phút và video dự phòng.

Kết thúc khi tất cả T01–T18 đạt; không còn lỗi nghiêm trọng ở đặt phòng, quyền và tiền; PF02 cho đúng một booking thành công; PF01 và PF03 có số liệu thật kèm kết luận; demo xuyên suốt trên bản triển khai; mọi thành viên giải thích được quy trình và các quyết định cốt lõi. Nếu chậm tiến độ, cắt P2 trước; không bỏ chống trùng, phân quyền hoặc xử lý ngoại lệ.

Đánh giá và trình bày

Nhận xét	Tác động và cách xử lý
Ưu: nghiệp vụ dễ hiểu	Khách đặt phòng, chọn phim, gọi món rồi thanh toán; phù hợp demo ngắn và có tính thực tế.
Ưu: đủ chiều sâu kỹ thuật	Khoảng chiếm dụng, giao dịch có khóa, ràng buộc loại trừ, idempotency, tách trạng thái tiền, số liệu đo tải; đủ nội dung Q&A.
Ưu: chia được cho 6 người	Các luồng tương đối rõ; review chéo giúp mọi người hiểu toàn hệ thống; có người tích hợp và người đo.
Nhược: dễ vượt phạm vi	Giữ chỗ tạm, giá giờ vàng, thanh toán online, nhiều cơ sở dễ kéo dài. Đưa vào hướng phát triển; không thêm sau mốc đóng băng.
Nhược: phụ thuộc tích hợp	Booking, món, điều chỉnh và thu tiền liên quan chặt. Một nguồn dữ liệu, trạng thái thống nhất, tích hợp từ tuần 2.
Giới hạn: không đặt cọc	Rủi ro khách không đến; giới hạn 3 booking tương lai và ghi NO_SHOW. Đặt cọc là hướng mở rộng.
Giới hạn: không công nợ	"Chưa thu" chỉ là danh sách để đối chiếu, không theo dõi thu hồi. Nêu đúng khả năng khi demo.

Kịch bản trình bày 6 phút

0:00–0:40: bài toán và phạm vi. **0:40–2:10:** khách tìm phòng, chọn phim/món và đặt. **2:10–3:20:** nhân viên chuẩn bị, check-in, món, một ngoại lệ (kết thúc sớm + điều chỉnh) và thu tiền. **3:20–4:30:** chống trùng: chạy PF02 trực tiếp hoặc chiếu kết quả, chỉ vào ràng buộc loại trừ. **4:30–5:20:** kiến trúc, dữ liệu, số liệu PF01/PF03. **5:20–6:00:** kết quả, hạn chế và mở rộng.

Nếu 5–7 phút gồm cả Q&A, rút phần nói xuống 4–5 phút. Cả sáu người tập trả lời và tập trình bày theo bốc thăm ít nhất hai lượt.

Câu hỏi dự kiến và ý trả lời

Câu hỏi	Ý chính
Vì sao không trùng phòng khi nhiều người đặt cùng lúc? 199 phản hồi 409 có phải lỗi?	Khóa dòng phòng trong giao dịch xếp hàng các yêu cầu cùng phòng; ràng buộc loại trừ trong PostgreSQL chặn ở tầng dữ liệu kể cả khi mã lỗi; PF02 chứng minh 1/200. 409 là xung đột nghiệp vụ dự kiến, tách khỏi 5xx/timeout.
Tiền được tính ở đâu, nếu khách sửa số trên trình duyệt?	Máy chủ tính từ giá đã chốt trong booking và món đã lưu; số từ trình duyệt chỉ để hiển thị (T09).
Vì sao không giữ chỗ tạm khi khách đang chọn?	Tránh chiếm chỗ và thêm cơ chế hết hạn; xung đột được báo rõ ở bước xác nhận kèm gợi ý giờ khác. Giữ chỗ là hướng mở rộng.
Vì sao chọn PostgreSQL và Prisma?	Bài toán cần giao dịch, khóa bản ghi và ràng buộc khoảng thời gian; Prisma cho migration và kiểu dữ liệu; phần đặc thù viết SQL trong migration.
Khóa tài khoản có hiệu lực ngay không?	Có; phiên lưu phía máy chủ nên khóa tài khoản xóa phiên, yêu cầu kế tiếp bị 401 (T17).
Khách rời phòng mà chưa trả thì sao?	Tách trạng thái sử dụng và thanh toán: phòng COMPLETED, tiền UNPAID, không vào doanh thu, có trong báo cáo chưa thu (T16).
Số liệu hiệu năng đo thế nào, có tin được không?	Locust trong LAN, cấu hình ghi rõ, dữ liệu sinh có seed, đo trước/sau cùng điều kiện, kết quả kèm lệnh chạy; nêu rõ đây là môi trường demo.

Nguồn tham khảo mô hình

Đã tham khảo ngày 07/09/2026: website T-Box (cafephimhd.com) và các bài giới thiệu café phim T-Box (loại phòng, giá theo giờ/combo, đặt phòng qua hotline, Messenger, Zalo). Nguồn chỉ để tham khảo mô hình; toàn bộ quy tắc phần mềm trong tài liệu là đề xuất cho đồ án.

Phụ lục A – SQL ràng buộc và giao dịch

Migration bổ sung sau khi Prisma sinh bảng

```
-- prisma/migrations/<timestamp>_constraints/migration.sql
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE "booking" ADD CONSTRAINT booking_time_check
  CHECK ("start_at" < "end_at" AND "end_at" <= "occupied_until");

ALTER TABLE "booking" ADD CONSTRAINT booking_no_overlap
  EXCLUDE USING gist ("room_id" WITH =, tstzrange("start_at", "occupied_until", '[]') WITH &&)
  WHERE ("status" IN ('CONFIRMED', 'IN_USE', 'COMPLETED'));

CREATE UNIQUE INDEX adjustment_one_current ON "adjustment" ("booking_id") WHERE ("is_current");

-- session theo schema connect-pg-simple (sid, sess, expire) + cột sinh để tìm phiên theo người dùng
ALTER TABLE "session" ADD COLUMN "user_id" integer GENERATED ALWAYS AS (NULLIF("sess" ->> 'userId',
  '')::integer) STORED;
CREATE INDEX session_user_idx ON "session" ("user_id"); -- khóa TK: DELETE FROM "session" WHERE "user_id" =
  $1
```

Chống gửi lặp ngoài giao dịch chính, rồi giao dịch tạo booking (Prisma, rút gọn; thu tiền dùng cùng khuôn mẫu)

```
const HOLDING = ['CONFIRMED', 'IN_USE', 'COMPLETED'], now = new Date();
// 0. Nhận khóa trong một giao dịch ngắn riêng; không bắt lỗi trùng rồi dùng tiếp giao dịch đã lỗi
const claimed = await prisma.$queryRaw`
  INSERT INTO idempotency_request (actor_id, key, request_hash, status)
  VALUES (${actorId}, ${key}, ${hash}, 'IN_PROGRESS') ON CONFLICT (actor_id, key) DO NOTHING RETURNING id`;
if (claimed.length === 0) {
  const prev = await prisma.idempotencyRequest.findUnique({ where: { actorId_key: { actorId, key } } });
  if (prev.requestHash !== hash) throw new ApiError(422, 'IDEMPOTENCY_KEY_REUSED');
  if (prev.status === 'IN_PROGRESS') {
    // bỏ dở quá 30 giây: xóa khóa, cho
    thử lại
    if (Date.now() - prev.createdAt > 30_000) { await deleteKey(prev.id); return retry(); }
    throw new ApiError(409, 'REQUEST_IN_PROGRESS');
  }
  return { code: prev.responseCode, body: prev.responseBody }; // thử lại hợp lệ: trả kết quả cũ
}
try {
  const booking = await prisma.$transaction(async (tx) => {
    // 1. Khóa phòng: các yêu cầu cùng phòng xếp hàng tại đây
    const [room] = await tx.$queryRaw`SELECT id, capacity, hourly_price_vnd, is_active
      FROM "room" WHERE id = ${roomId} FOR UPDATE`;
    if (!room?.is_active) throw new ApiError(422, 'ROOM_INACTIVE');
    const quote = await buildQuote(tx, room, input); // 2. quy tắc thời gian, sức chứa, phim, món; tính
    tiền từ giá trong DB
    // 3. Kiểm tra giao nhau để trả 409 kèm gợi ý (EXCLUDE vẫn chặn nếu bước này sót)
    const clash = await tx.booking.findFirst({ where: { roomId, status: { in: HOLDING },
      startAt: { lt: quote.occupiedUntil }, occupiedUntil: { gt: quote.startAt } } });
    if (clash) throw new ApiError(409, 'ROOM_TAKEN', { suggest: await nextFree(tx, roomId, quote) });
    // 4. Giới hạn 3 booking tương lai: khóa dòng user để đếm chính xác
    if (customerId) {
      await tx.$queryRaw`SELECT id FROM "user" WHERE id = ${customerId} FOR UPDATE`;
      const n = await tx.booking.count({ where: { customerId, status: 'CONFIRMED', startAt: { gt: now } } });
      if (n >= 3) throw new ApiError(422, 'BOOKING_LIMIT');
    }
    const b = await tx.booking.create({ data: quote.toBookingRow() });
    await tx.foodOrder.create({ data: quote.toFoodOrderRow(b.id) });
    await tx.bookingStatusHistory.create({ data: { bookingId: b.id, field: 'status', newValue: 'CONFIRMED',
      actorId } });
    return b;
  }, { timeout: 5000 });
  await prisma.idempotencyRequest.update({ where: { actorId_key: { actorId, key } },
    data: { status: 'DONE', responseCode: 201, responseBody: booking } });
  return { code: 201, body: booking };
} catch (e) {
  await deleteKey(actorId, key); // thất bại: xóa khóa để cho phép thử lại với cùng khóa
  throw mapDbError(e); // 23P01, 23505 -> 409; ApiError giữ nguyên; còn lại -> 500 có mã
  tra cứu
}
```

Phụ lục B – Kịch bản Locust mẫu

Hai file trong thư mục perf/: locustfile.py (người dùng ảo) và ramp.py (hình dạng tải cho PF03, chỉ nạp khi cần). Mỗi lệnh chỉ định rõ lớp người dùng, vì Locust mặc định chạy mọi lớp có trong file. Ngày thử là ngày tính lúc chạy (mặc định hôm nay + 7, luôn trong cửa sổ 14 ngày); tham số API đúng hợp đồng trang 13 (date, startTime, duration, guests).

```
PF01: locust -f perf/locustfile.py Customer --headless -u 50 -r 10 -t 5m --tags search --host
http://api.local --csv perf/reports/pf01
PF02: locust -f perf/locustfile.py Contender --headless -u 200 -r 200 -t 2m --host
http://api.local --csv perf/reports/pf02
PF03: locust -f perf/locustfile.py,perf/ramp.py Customer --headless --tags journey --host
http://api.local --csv perf/reports/pf03
```

```
# perf/locustfile.py
import os, random, uuid
from datetime import date, timedelta
from gevent.event import Event
from locust import HttpUser, task, between, constant, tag, events

ACCOUNTS = [(f"perf{i}@demo.local", "Perf#12345") for i in range(1, 301)] # 300 tài khoản seed
PF_DATE = os.getenv("PF_DATE") or (date.today() + timedelta(days=7)).isoformat() # trong 14 ngày tới
TARGET = {"roomId": 1, "date": PF_DATE, "startTime": "19:00", "duration": 120} # phòng/giờ seed để trống
cho PF02
SUMMARY = {"201": 0, "409": 0, "422": 0, "other": 0}
ALL_READY = Event() # rào chắn cho PF02

def login(client, email, pw):
    client.post("/api/auth/login", json={"email": email, "password": pw}, name="POST /auth/login")

def book(client, payload, name, allow_422):
    """201 và 409 luôn là phản hồi đúng; 422 chỉ đúng khi kịch bản chấp nhận (PF03: quá 3 booking)."""
    with client.post("/api/bookings", json=payload, headers={"Idempotency-Key": str(uuid.uuid4())},
                    name=name, catch_response=True) as r:
        if r.status_code == 201: SUMMARY["201"] += 1; r.success()
        elif r.status_code == 409: SUMMARY["409"] += 1; r.success() # xung đột dự kiến
        elif r.status_code == 422 and allow_422: SUMMARY["422"] += 1; r.success()
        else: SUMMARY["other"] += 1; r.failure(f"unexpected {r.status_code}")

class Customer(HttpUser):
    """PF01 (--tags search) và PF03 (--tags journey)."""
    wait_time = between(2, 5)

    def on_start(self):
        login(self.client, *random.choice(ACCOUNTS))

    def _search(self):
        q = {"date": (date.today() + timedelta(days=random.randint(0, 13))).isoformat(),
            "startTime": random.choice(["09:00", "11:30", "14:00", "16:30", "19:00", "20:30"]),
            "duration": 120, "guests": 2}
        with self.client.get("/api/rooms/availability", params=q, name="GET /rooms/availability",
                            catch_response=True) as r:
            if r.status_code != 200:
                r.failure(f"status {r.status_code}"); return q, []
            return q, (r.json().get("data") or [])

    @tag("search")
    @task
    def search_rooms(self):
        self._search()
```

Phụ lục B – Kịch bản Locust mẫu (tiếp)

```
# perf/locustfile.py (tiếp)
@tag("journey")
@task
def journey(self):
    q, rooms = self._search()
    if not rooms:
        return # không có phòng trống: dừng lượt này
    room = random.choice(rooms) # dùng đúng phòng do kết quả tìm trả về
    self.client.get(f"/api/rooms/{room['id']}", name="GET /rooms/:id")
    if random.random() < 0.10: # 10% lượt kết thúc bằng đặt phòng
        book(self.client, {"roomId": room["id"], **q, "contactName": "Perf", "contactPhone":
"0900000000"},
            name="POST /bookings", allow_422=True)

class Contender(HttpUser):
    """PF02: mỗi người một tài khoản riêng; đăng nhập xong chờ đủ 200 người rồi cùng gửi một yêu cầu."""
    wait_time = constant(0)
    seq = 0

    def on_start(self):
        Contender.seq += 1
        login(self.client, *ACCOUNTS[Contender.seq % len(ACCOUNTS)])

    @task
    def contend(self):
        ALL_READY.wait() # rào chắn: chỉ mở khi spawning_complete
        book(self.client, **TARGET, "guests": 2, "contactName": "Perf", "contactPhone": "0900000000"},
            name="POST /bookings (contention)", allow_422=False) # 422 ở đây là dữ liệu thừa sai
        self.stop(force=True)

@events.spawning_complete.add_listener
def open_barrier(user_count, **kw):
    ALL_READY.set() # đủ 200 người mới cùng gửi

@events.quitting.add_listener
def print_summary(environment, **kw):
    print("BOOKING RESULTS:", SUMMARY) # PF02 mong đợi: 201 = 1, 409 = 199, 422 = 0, other = 0
```

```
# perf/ramp.py (PF03: 0->100->200->300 trong 3 phút, giữ 300 trong 3 phút, giảm về 50 trong 2 phút)
from locust import LoadTestShape

class RampShape(LoadTestShape):
    stages = [(60, 100, 5), (120, 200, 5), (180, 300, 5), (360, 300, 5), (480, 50, 10)] # (giây, users,
spawn/giây)
    def tick(self):
        t = self.get_run_time()
        for limit, users, rate in self.stages:
            if t < limit:
                return (users, rate)
        return None
```

Sau PF02, chạy `SELECT count(*) FROM booking WHERE room_id = 1 AND status IN ('CONFIRMED', 'IN_USE', 'COMPLETED') AND start_at = '<PF_DATE 19:00 giờ VN>';` kết quả phải bằng 1. Ghi đầu ra Locust (bảng thống kê, CSV, dòng BOOKING RESULTS) và câu truy vấn vào báo cáo. Khôi phục cơ sở dữ liệu từ bản seed trước mỗi lần đo để các lần đo so sánh được.

Phụ lục C – Cấu trúc giao diện nhiều trang và chủ sở hữu

Mỗi màn hình là một file HTML kèm một module TypeScript cùng tên; Vite liệt kê mọi HTML làm điểm vào. Thư mục `shared/` do người 1 giữ; không copy header/nav vào từng trang mà gọi `mountLayout()`; trang nội bộ gọi `requireRole()` ở dòng đầu; không gọi `fetch` trực tiếp mà dùng `shared/api.ts`. Quản lý dùng lại mọi trang trong `staff/`.

frontend/	
├ vite.config.ts	mọi *.html là điểm vào; dev proxy /api → Express (cookie phiên cùng origin)
├ shared/	Dương: api.ts (fetch có cookie, bóc {ok,data,error}, 401 → login), auth.ts
(requireRole),	
	layout.ts (header theo vai trò, toast, tải/rỗng/lỗi), format.ts (100.000đ,
dd/mm/yyyy), styles.css	
	movie-picker.ts – Thành Lê: ô chọn phim (tìm tên, lọc thể loại, vừa gói,
Xem thêm)	
├ index.html	Chúc: trang chủ + form tìm phòng
├ rooms.html	Chúc: kết quả tìm phòng (API đo ở PF01)
├ room.html?id=	Chúc: chi tiết phòng (ảnh, tiện nghi, chính sách)
├ login.html, register.html	Dương
├ booking.html?roomId&date&startTime&duration&guests	
	Hải Anh: 4 bước trong MỘT trang; bước 2 phim (widget Thành Lê), bước 3 món
(widget Sơn)	
├ my-bookings.html	Hải Anh: sắp tới / đã kết thúc
├ booking-view.html?code=	Hải Anh: chi tiết, hủy; đổi phim (Thành Lê)
├ staff/	
	Dương: trang "hôm nay"
	Hải Anh: lịch ngày theo phòng + "đã qua giờ nhưng chưa xử lý" (EX06)
	Hải Anh: khách tại quầy (UC04)
	Hải Anh: check-in, NO_SHOW, hủy, sử dụng không check-in; nút sang checkout
	Thành Lê: READY (khóa lạc quan) / UNAVAILABLE
	Sơn: đơn món theo trạng thái, thêm món cho booking IN_USE
	Công Thành: hóa đơn, kết thúc sớm + món dở, đề nghị giảm/miễn, thu tiền
├ admin/	
├ index.html	Dương
├ employees.html	Dương: tạo/khóa nhân viên (T17)
├ rooms.html	Chúc: phòng và giá (BR06, BR08)
├ movies.html	Thành Lê (MOV07)
├ menu.html	Sơn
├ adjustments.html	Công Thành: hàng đợi duyệt (dùng được trên điện thoại)
├ reports.html	Công Thành: booking theo ngày, doanh thu, giờ phòng, chưa thu và miễn

Bộ nền dùng chung (đã sinh kèm đặc tả này)

Repo `movie-cafe-booking` gồm: `backend/` (Express 5, Prisma 7 + adapter pg, PostgreSQL 16; `schema.prisma` đủ mọi bảng trang 11; migration 0001 bảng và 0002 ràng buộc theo Phụ lục A; seed 8 phòng, 51 phim thật, 15 món, 4 tài khoản, 10 booking đủ trạng thái; script `db:import-movies` nhập kho phim từ The Movies Dataset (Kaggle/TMDB); `core/` gồm phiên, phân quyền, chuẩn lỗi, ảnh xạ 23P01/23505 → 409, chống gửi lặp, quy tắc thời gian; sáu module có service + route chạy được; 22 kiểm thử Vitest + Supertest cho T01–T06, T09–T12, T14–T18 và danh mục phim), `frontend/` (22 trang ở trên, ô chọn phim tìm kiếm/phân trang dùng chung, chạy xuyên suốt với dữ liệu mẫu), `perf/` (`locustfile.py`, `ramp.py`), `docs/` (API.md, CONTRACTS.md), Docker Compose và CI. Hướng dẫn cài đặt và điểm bắt đầu của từng người ở README.md.

Thay đổi hai chỗ so với bản 1.5: tiền tố mã booking là MC- (thay CB-); tên thư mục dự án chưa gắn tên thương hiệu vì nhóm chưa chốt tên hệ thống.